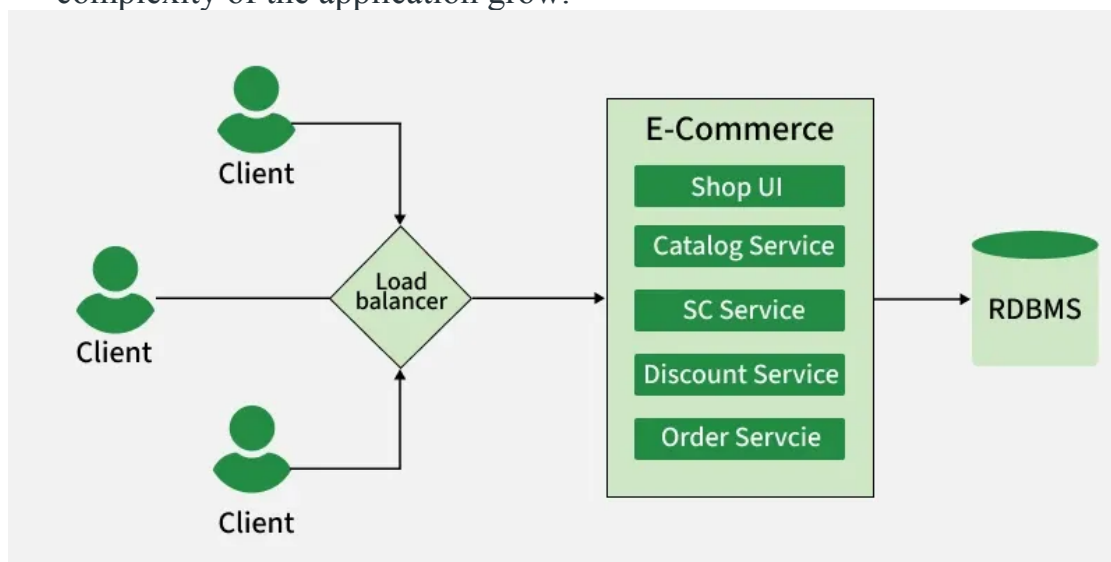


Two common ways to structure software are called monolithic and microservices architectures. Let us see how these two approaches differ and when you might choose one over the other.

Monolithic Architecture

Software is traditionally designed using a [monolithic architecture](#), in which the entire program is constructed as a single, indivisible unit.

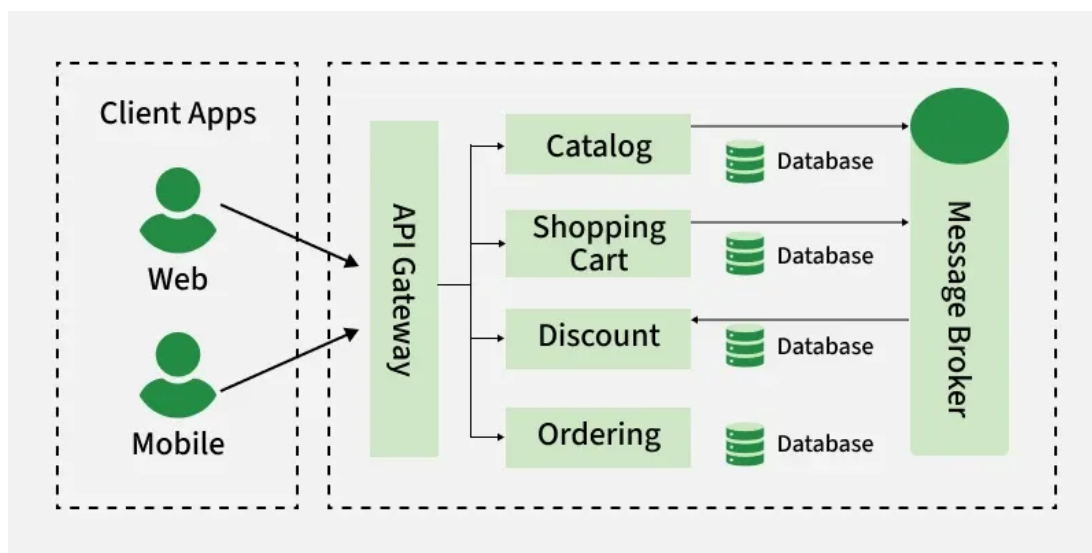
- This means that any changes or updates to the application require modifying and redeploying the entire monolith.
- Monolithic architectures are often characterized by their simplicity and ease of development, especially for small to medium-sized applications.
- However, they can become complex and difficult to maintain as the size and complexity of the application grow.



Microservices Architecture

A [microservices architecture](#) results in an application designed as a set of small, independent services. Each one represents a business capability in itself. The services loosely couple with one another and communicate over the network, typically making use of lightweight protocols such as HTTP or messaging queues.

- Each service is responsible for a single functionality or feature of the application and can be developed, deployed, and scaled independently.
- The Microservice architecture has a significant impact on the relationship between the application and the database.



Monolith Vs. Microservices

Scenario	Monolith	Microservices
Small, simple application	✓	✗
Limited development team resources	✓	✗
Low operational complexity	✓	✗
Frequent changes are not expected	✓	✗
Focus on rapid deployment & iteration	✗	✓
High scalability & fault isolation needed	✗	✓
Leveraging different technology stacks	✗	✓
Dynamically evolving product or market	✗	✓

Differences between Monolithic and Microservices Architecture

Below are the differences the Monolithic and Microservice architecture:

Monolithic Architecture	Microservice Architecture
Single-tier architecture	Multi-tier architecture
Built as one large application with tightly coupled components	Composed of small, loosely coupled services components
Deployed as a single unit	Individual services can be deployed independently

Horizontal scaling can be challenging	Easier to scale horizontally
Development is simpler initially	Development is more complex due to multiple services
Technology stack choices are usually limited	Freedom to choose the best technology for each service
Entire application may fail if a part fails	Individual services can fail without affecting others
Easier to maintain due to its simplicity	Requires more effort to manage multiple services
Less flexible as all components are tightly coupled	More flexible as components can be developed, deployed, and scaled independently
Communication between components is faster	Communication may be slower due to network calls

Part 2

A monolithic application is built as a single unified unit while a microservices architecture is a collection of smaller, independently deployable services. Which one is right for you? It depends on a number of factors.

In 2009 Netflix faced growing pains. Its infrastructure couldn't keep up with the demand for its rapidly growing video streaming services. The company decided to migrate its IT infrastructure from its private data centers to a public cloud and replace its monolithic architecture with a microservices architecture. The only problem was, the term "microservices" didn't exist and the structure wasn't well-known.

Netflix became one of the first high-profile companies to successfully migrate from a monolith to a cloud-based microservices architecture. It won the [2015 JAX Special Jury award](#) in part due to this new infrastructure that internalized DevOps. Today, Netflix has more than a thousand microservices that manage and support separate parts of the platform, while its engineers deploy code frequently, sometimes thousands of times each day.

Netflix was an early pioneer in what has become increasingly common today: transitioning from a monolith architecture to a microservices architecture.

What is a monolithic architecture?

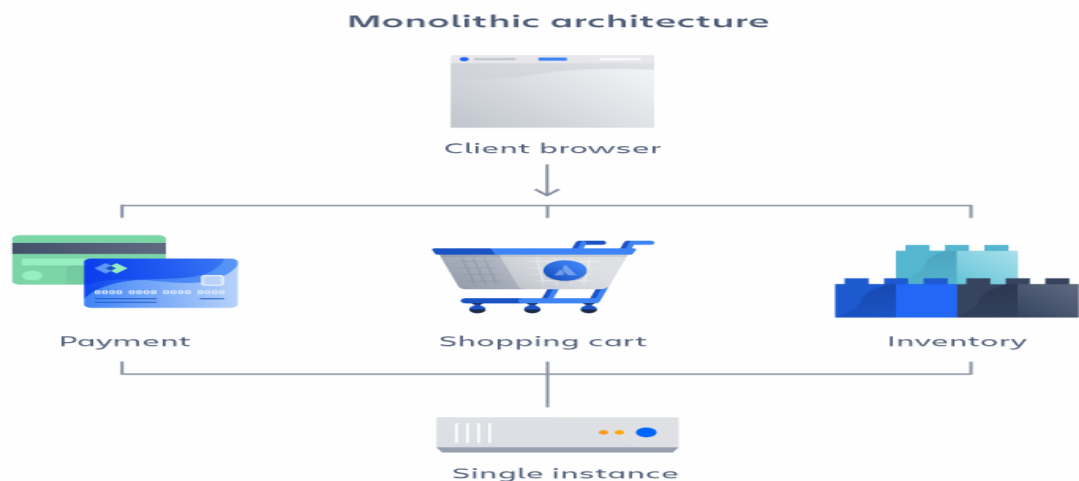
A monolithic architecture is a traditional model of a software program, which is built as a unified unit that is self-contained and independent from other applications.

The word "monolith" is often attributed to something large and glacial, which isn't far from the truth of a monolith architecture for software design.

A monolithic architecture is a singular, large computing network with one code base that couples all of the business concerns together. To make a change to this sort of application requires updating the entire stack by

accessing the code base and building and deploying an updated version of the service-side interface. This makes updates restrictive and time-consuming.

Monoliths can be convenient early on in a project's life for ease of code management, cognitive overhead, and deployment. This allows everything in the monolith to be released at once.



Advantages of a monolithic architecture

Organizations can benefit from either a monolithic or microservices architecture, depending on a number of different factors. When developing using a monolithic architecture, the primary advantage is fast development speed due to the simplicity of having an application based on one code base.

The advantages of a monolithic architecture include:

Easy deployment – One executable file or directory makes deployment easier.

Development – When an application is built with one code base, it is easier to develop.

Performance – In a centralized code base and repository, one API can often perform the same function that numerous APIs perform with microservices.

Simplified testing – Since a monolithic application is a single, centralized unit, end-to-end testing can be performed faster than with a distributed application.

Easy debugging – With all code located in one place, it's easier to follow a request and find an issue.

Disadvantages of a monolithic architecture

As with the case of Netflix, monolithic applications can be quite effective until they grow too large and scaling becomes a challenge. Making a small change in a single function requires compiling and testing the entire platform, which goes against the agile approach today's developers favor. The disadvantages of a monolith include:

Slower development speed – A large, monolithic application makes development more complex and slower.

Scalability – You can't scale individual components.

Reliability – If there's an error in any module, it could affect the entire application's availability.

Barrier to technology adoption – Any changes in the framework or language affects the entire application, making changes often expensive and time-consuming.

Lack of flexibility – A monolith is constrained by the technologies already used in the monolith.

Deployment – A small change to a monolithic application requires the redeployment of the entire monolith.

What are microservices?

A microservices architecture, also simply known as microservices, is an architectural method that relies on a series of independently deployable services.

These services have their own business logic and database with a specific goal. Updating, testing, deployment, and scaling occur within each service. Microservices decouple major business, domain-specific concerns into separate, independent code bases.

Microservices don't reduce complexity, but they make any complexity visible and more manageable by separating tasks into smaller processes that function independently of each other and contribute to the overall whole.

Adopting microservices often goes hand in hand with DevOps, since they are the basis for [continuous delivery](#) practices that allow teams to adapt quickly to user requirements.

Microservice architecture



Advantages of microservices

Microservices are by no means a silver bullet, but they solve a number of problems for growing software and companies. Since a microservices architecture consists of units that run independently, each service can be developed, updated, deployed, and scaled without affecting the other services.

Software updates can be performed more frequently, with improved reliability, uptime, and performance. We went from pushing updates once a week, to two to three times a day.

As Atlassian grows, microservices enable us to scale teams and geographic locations more reliably by splitting along lines of service ownership. Before we started Vertigo, Atlassian had five different development centers around the world. These distributed teams were constrained by a centralized monolith and we needed to support them in an autonomous fashion. Microservices allow us to do so.

The benefits of Vertigo include increased deployment speed, disaster recovery, reduced cost, and higher performance. This allows us to get to our target faster while delivering more incremental value to customers along the way.

Plus, more generally, microservices make it easier for teams to update code and accelerate release cycles with [continuous integration and continuous delivery \(CI/CD\)](#). Teams can experiment with code and roll back if something goes wrong.

In short, the advantages of microservices are:

Agility – Promote agile ways of working with small teams that deploy frequently.

Flexible scaling – If a microservice reaches its load capacity, new instances of that service can rapidly be deployed to the accompanying cluster to help relieve pressure. We are now multi-tenant and stateless with customers spread across multiple instances. Now we can support much larger instance sizes.

Continuous deployment – We now have frequent and faster release cycles. Before we would push out updates once a week and now we can do so about two to three times a day.

Highly maintainable and testable – Teams can experiment with new features and roll back if something doesn't work. This makes it easier to update code and accelerates time-to-market for new features. Plus, it is easy to isolate and fix faults and bugs in individual services.

Independently deployable – Since microservices are individual units they allow for fast and easy independent deployment of individual features.

Technology flexibility – Microservice architectures allow teams the freedom to select the tools they desire.

High reliability – You can deploy changes for a specific service, without the threat of bringing down the entire application.

Happier teams – The Atlassian teams who work with microservices are a lot happier, since they are more autonomous and can build and deploy themselves without waiting weeks for a pull request to be approved.

Disadvantages of microservices

When we moved from a small number of monolithic codebases to many more distributed systems and services powering our products, unintended complexity arose. We initially struggled to add new capabilities with the same velocity and confidence as we had done in the past. Microservices can add increased complexity that leads to development sprawl, or rapid and unmanaged growth. It can be challenging to determine how different components relate to each other, who owns a particular software component, or how to avoid interfering with dependent components.

With Vertigo, we built a common functionality that would power our existing products and future products we acquire and build. If you are a single product company, microservices may not be necessary.

The disadvantages of microservices can include:

Development extension – Microservices add more complexity compared to a monolith architecture, since there are more services in more places created by multiple teams. If development sprawl isn't properly managed, it results in slower development speed and poor operational performance.

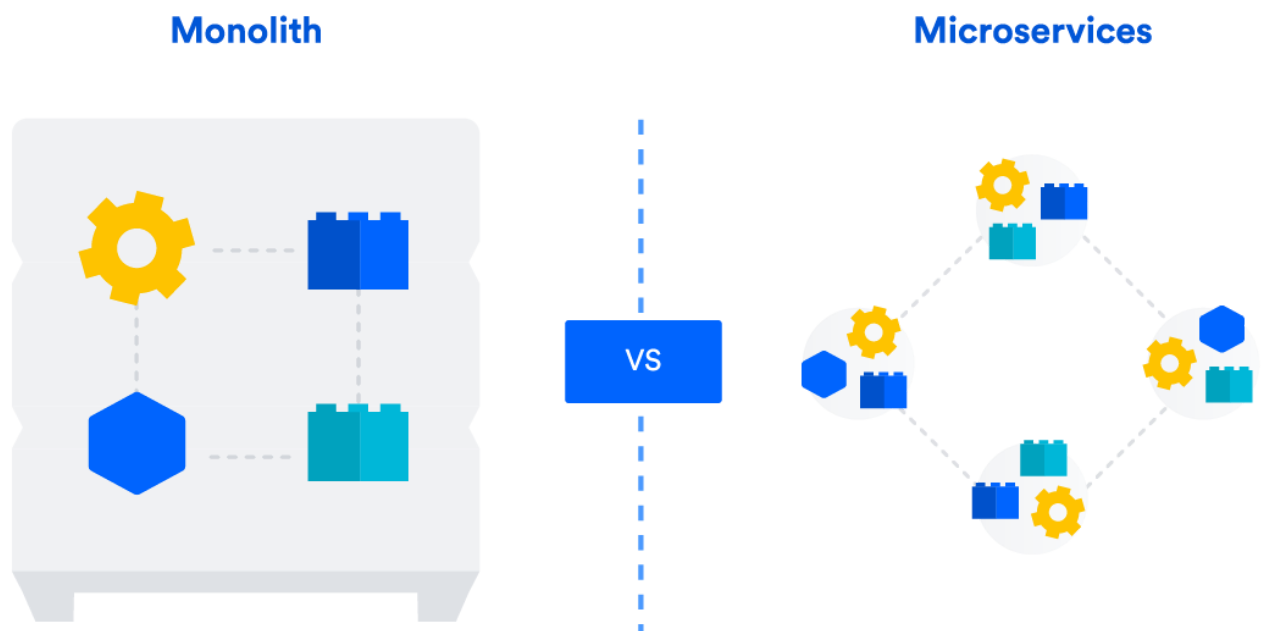
Exponential infrastructure costs – Each new microservice can have its own cost for test suite, deployment playbooks, hosting infrastructure, monitoring tools, and more.

Added organizational overhead – Teams need to add another level of communication and collaboration to coordinate updates and interfaces.

Debugging challenges – Each microservice has its own set of logs, which makes debugging more complicated. Plus, a single business process can run across multiple machines, further complicating debugging.

Lack of standardization – Without a common platform, there can be a proliferation of languages, logging standards, and monitoring.

Lack of clear ownership – As more services are introduced, so are the number of teams running those services. Over time it becomes difficult to know the available services a team can leverage and who to contact for support.



Microservices Design Principles

- Ever wondered, how big applications like Amazon, Netflix, etc. never face any major faults and are always up and running? Is it possible for such big and complex pieces of code to always be error and exceptions free? Well, that's very hard to believe, right? But the answer is NO. They do face faults but are always running and serving millions of users without the services getting down. And that's where **Microservices** come to play.



In this article, we'll be going through the **microservices design principles that every developer should follow** in order to run an application efficiently. So, before that let's understand what are microservices.

What are Microservices?

When a large application is broken down into a number of small, independent services that can communicate with each other over a network, this approach is known as Microservices.

These services can each be created, deployed, and scaled independently. Each microservice carries out a particular business activity and interacts with other microservices via clear APIs.

This is why even if a piece of code fails, only that particular microservice gets affected making no major change to the whole application and that is how applications like Amazon and Netflix continue to serve their customers uninterrupted.

Microservices can be developed in any programming language. They communicate with each other using language-neutral Application Programming Interfaces (APIs) such as Representational State Transfer (REST).

For developing an optimal microservice architecture, you need to follow some design principles.

Microservices Design Principles

1. Independent and Autonomous/Self-governing services
1. API aggregation
1. Flexibility
1. Scalability
1. Constant monitoring
1. Failure Isolation/ Failure resilience
1. Realtime Load balancing
1. Inclusion of DevOps
1. Versioning
1. Availability

Let's take a deeper dive.

1. Independent and Autonomous/Self-Governing Services

Every microservice should be self-contained and should operate independently of all the other services in an application. *Each service should have its resources to run including separate databases and business logic without relying on other services to perform any of its functions.*

The main reason behind this principle is to ensure that each service can be **developed, tested, as well as deployed independently** without

affecting the other part of the system. This also follows the idea of Decentralization which refers to the idea of not giving control to any central unit/ service, instead, each service should be responsible for its actions as well as should have the liberty to act independently.

Let us consider Amazon's system, so in this system, each microservice would have its specific business function - product catalog, final cart, payment portal, etc. and every function would have its database as well as its performing logic without depending upon other services. Eventually, this will help in the scalability, resiliency, and maintainability of each microservice.

2. API Aggregation

A developer should be able to design microservices in such a manner that they can communicate throughout the system with other microservices without having a programming language barrier. Most of the time different microservices belonging to the same system may be written in different programming languages but as the intercommunication among the microservices is important this is achieved with the help of **API aggregation** using different architectural approaches such as REST. This principle emphasizes the use of well-defined and consistent APIs to enable communication between different microservices in the system.

Suppose the frontend part of Amazon is written in a particular language and its backend server is written in a different language then, To communicate between themselves, APIs are needed which help in fetching and sending, requests and responses.

3. Flexibility

The ability of a microservice to change or be changed easily according to the circumstances or the need of the time is referred to as the flexibility of a microservice. Assuming you added a feature that was criticized and was not successful, now that it needed to be taken out of the service, this can be done easily when you have made a flexible microservice. This principle **makes your project more adaptable to possible future changes.**

4. Scalability

It is a design principle that **enables your application to get modified according to the increasing or decreasing traffic, data, and complexity without affecting the performance of the system.**

Assume the festive season is going on and there is a high demand for supply due to which the level of traffic on the application drastically increases. At that time scalability plays its role by modifying the capacities of all the microservices needed such as databases, servers, etc. to serve the increasing traffic with high performance.

There are several ways to achieve scalabilities such as **Service partitioning**, **Load balancing**, **Horizontal scaling**, and **Caching**.

5. Constant monitoring

When numerous microservices are working in synchronization to successfully run an application there is a high possibility of faults occurring throughout the system. As the architecture becomes more complex, it becomes a tedious task to find and resolve these issues. To avoid this, constant monitoring is required so that the **faults can be resolved at the earliest and your application can run smoothly**.

Constant monitoring can be achieved by using these methods-

- **Logging and metrics**
- **Distributed tracing**
- **Health checks**
- **Alerting and notifications**

6. Failure Isolation/ Failure Resilience

As a developer, you should always be prepared to encounter failures in your application, and to minimize these failures you should include ***fault-tolerant approaches during the designing of microservices***. This principle helps to minimize the impact of failures on the whole system due to the failure of a particular system.

There are several techniques to make your application fault tolerant:

- **Include Redundancy** in order to have multiple instances present at the time of failure.
- **Separation of Services** in order to be isolated in case of failures.
- The microservices should **Gracefully Degrade** instead of crashing the whole service.
- Each microservice should have **Circuit Breakers** in order to detect and isolate the failures.

7. Realtime Load Balancing

Sometimes it may happen when a client sends the request to the server then the database has to retrieve the data from multiple microservices at the same time. At that point, the Load balancer comes into the picture and it defines how much amount of the **Central Processing Unit or GPU** is to be used for a particular service to fetch the data and finally how the client request should be passed.

This whole process needs to be done in "Real-time" so that the client does not need to wait for the results.

Suppose you are at the payment portal, purchasing something on Amazon. At that time Amazon application is fetching multiple data like UPI integration, CC database, etc. With the help of real-time load balancing it gets easier for the application to perform multiple actions in less time.

8. Inclusion of DevOps

First of all, you need to know what is **DevOps**. DevOps is a **set of techniques applied that is used to provide continuous support in order to collaborate, communicate and automate the development and operation with the help of operations teams**. The inclusion of DevOps in a project promotes communication and collaboration between different microservices that in the end improves the efficiency and effectiveness of the software development process. DevOps helps microservices to achieve greater speed, high flexibility, and agility. DevOps inclusion can be done using several services such as Automated Testing, Continuous Delivery, and Development as well as Continuous Monitoring that leads to faster time-to-market and best-quality software. **Docker** and **Kubernetes** are two common DevOps tools used for managing microservices.

9. Versioning

Isn't it shocking for you that the applications like Amazon, and Netflix are always up to date and compatible with the latest software updates of the devices? This is possible through Versioning. Versioning is a term used for *updating to the latest versions of the services according to the latest tech stacks that are being used in present*.

Versioning helps to **manage changes in the services over time and update them to the latest ones**. This results in minimizing the disruption to the existing clients using the services. The versioning process includes updating version numbers, service interface, request, and response through URL versioning and the same in Header Versioning.

10. Availability

When businesses are serving worldwide then there are no chances that can be taken with downtime. Every minute can be proved to serve big losses if the services are down, thus **availability is a very important design principle** which states that all the services should be developed in such a manner that the services are up and working 24x7 or if not possible, they can at least be available for the maximum amount of time. Availability straightway means the presence of microservices **24x7**.

Conclusion

These are the **top 10 design principles a developer should keep in mind** while designing any microservice. All the design principles revolve around how the best solution available all the time can be served to the end customers without any failure or major downtimes. Everything from communication among the microservices to serving the end product of the application to the customers is smoothly possible only because of the usage of the microservice architecture. By adopting these **design principles and embracing a microservices architecture**, organizations can build software systems that can adapt and thrive in today's fast-paced, constantly changing business environment.